AUTO-INCIDENT TO AGILE TICKET PIPELINE (DEVOPS INCIDENT WATCHDOG)

Continuous Site Reliability Engineering (SRE), Cloud Forensics, and Zero-Touch Agile Sprint Backlog Automation

Student Name:	S Hemal	**Course Title:**	Agile Development Process & DevOps
Registration Number:	24MIS0050	**Institution:**	Vellore Institute of Technology (VIT), Vellore Campus
Cloud Environment:	AWS (ap-southeast-2 Sydney)	**Account ID:**	401537512022
Version Control:	GitHub (hemalexe/devopss-incident-watchdog)	**Pipeline Status:**	CI/CD Automated Testing (Passing)

1. EXECUTIVE SUMMARY & PROBLEM FORMULATION

In modern cloud architectures, a significant operational divide exists between Site Reliability Engineering (monitoring & telemetry) and Agile Project Management (sprint backlog execution). In high-throughput consumer services—such as quick-commerce platforms (Zepto, Blinkit) and promotional retail environments—unhandled exceptions frequently occur during peak promotional flash sales when high concurrency exhausts database records or inventory pools to zero.

Under traditional operational workflows, when an unhandled server crash occurs (HTTP 500), the incident lifecycle is severely impeded by manual intervention:

1. The production outage logs are dispersed across proprietary cloud log streams (e.g., AWS CloudWatch).
2. Support engineers must manually extract raw stack traces, reproduce the defect locally, and manually transcribe the issue into project management software (e.g., Jira or GitHub Issues).
3. This manual triaging introduces a high Mean-Time-to-Acknowledge (MTTA), risks transcript omissions, and divorces operational metrics from the development team's sprint backlog.

The **DevOps Incident Watchdog** addresses this friction by introducing an autonomous, event-driven closed-loop architecture. Runtime exceptions in cloud production microservices are filtered instantaneously by Amazon CloudWatch, triaged by an AWS Lambda Watchdog, archived in Amazon S3 as forensic audit records, and automatically converted into structured Agile Bug User Stories on GitHub Issues complete with explicit Acceptance Criteria. Resolution is achieved via Smart Commit

integration (`Fixes #1`), triggering automated regression testing via GitHub Actions CI/CD and transitioning the ticket to Done without human administrative overhead.

2. TECHNICAL ARCHITECTURE & COMPONENT INTERACTION

The architecture encompasses three functional tiers: the **Compute Tier** (target application running on Python 3.12 Flask via serverless WSGI), the **Observability & Forensics Tier** (Amazon CloudWatch and Amazon S3), and the **Agile DevOps Automation Tier** (AWS Lambda Watchdog and GitHub Actions CI/CD).

[PASTE ARCHITECTURE DIAGRAM HERE]

INSTRUCTIONS TO IMPORT NON-AI ENGINEERING DIAGRAM:
1. Open your browser and go to: `https://app.diagrams.net` (Draw.io).
2. Click **File > Open From > Device** and select the file: `architecture_diagram_drawio.xml` (saved in your project root).
3. The diagram will open with official AWS shapes, clear directional flow arrows, and component boundaries.
4. Click **File > Export as > PNG image** (set DPI to 300 for high resolution).
5. Copy that image and paste it directly into this box.

Figure 1: Event-Driven System Architecture and Zero-Touch Telemetry Pipeline

Component	Underlying Technology	Operational Role in Pipeline
Client Interface	HTML5 / CSS3 / JavaScript Web UI	Simulates customer ordering and dark-store fulfillment for locations (Default: VIT Vellore Campus). Handles zero-stock edge cases gracefully.
Monitored App	Python 3.12 / Flask WSGI (AWS Lambda)	Hosts production catalog and checkout calculation endpoints. Emits structured JSON logs to standard output.
Log Metric Filter	Amazon CloudWatch Logs	Monitors log group `/aws/lambda/watchdog-target-app` using metric filter pattern `[INCIDENT_DETECTED]`.
Watchdog Triager	AWS Lambda (Serverless Python)	Executes asynchronously upon CloudWatch trigger; decodes base64 gzipped payload, extracts traceback, and mints unique Incident ID.
Forensic Vault	Amazon Simple Storage Service (S3)	Stores immutable JSON crash dump file at `s3://devops-incident-watchdog-`

Component	Underlying Technology	Operational Role in Pipeline
		`logs-401537512022/incidents/...`.
Agile Kanban Board	GitHub Issues REST API v3	Maintains sprint backlog. Receives automated Bug User Story with Definition of Done acceptance criteria.
CI/CD Engine	GitHub Actions CI/CD Runner	Executes 9 automated unit regression tests upon developer commit and deploys updated code to AWS.

3. DETAILED PROCESS WALKTHROUGH & VERIFICATION SNAPSHOTS

The following chronological phases detail the execution of an incident from initiation through resolution:

Step 1: Baseline Application Health & Proactive Telemetry

Prior to any incident, the target microservice exposes proactive telemetry via the `/health` endpoint. Site Reliability Engineering standards mandate periodic health checks from Route 53 or synthetic monitors to verify service availability and uptime.

[PASTE SNAPSHOT 1: APPLICATION HEALTH CHECK]

HOW TO CAPTURE:
- Open your browser to: `http://127.0.0.1:5000/health`
- Capture the browser screen displaying the JSON response: `{"http_status": 200, "service": "watchdog-target-app", "status": "healthy", "uptime_seconds": ...}`.
- Paste the screenshot in this box.

Figure 2: Proactive Health Monitoring Endpoint Telemetry (HTTP 200 OK)

Technical Assessment: The service reports HTTP 200 with JSON payload confirming uptime and operational parameters. This confirms the baseline environment is functional.

Step 2: Customer Storefront & Multi-Address Dark Store Delivery Selection

The customer storefront simulates a high-speed quick-commerce electronics store (FlashMart Tech). The interface allows customers to select delivery addresses across university campus hubs and residential centers:

- **VIT Vellore Campus** (Men's Hostel Q-Block, Katpadi - 632014) [Default]
- **Gandhi Nagar, Vellore** (Near Katpadi Railway Station - 632006)
- **Anna Nagar West, Chennai** (2nd Avenue, Roundtana - 600040)
- **Indiranagar, Bangalore** (100ft Road, Stage 2 - 560038)

[PASTE SNAPSHOT 2: CUSTOMER STOREFRONT VIEW]

HOW TO CAPTURE:
- Open: `http://127.0.0.1:5000/` in your browser.
- Ensure **"1. Customer View"** is selected in the top bar.
- Capture the header showing: *"Deliver to: VIT Vellore Campus (Men's Hostel Q-Block,*

Katpadi • 10 Mins)" and the 4 products.
- Paste the screenshot in this box.

Figure 3: Customer Quick-Commerce Storefront Configured for VIT Vellore Campus

Step 3: Promotional Flash Sale Inventory Depletion & Location Stockout Handling

The customer attempts to purchase the promotional deal item: **OnePlus 11R 5G (16GB RAM, 256GB Storage)** priced at ₹34,999. In this scenario, high flash sale concurrency has exhausted dark store stock to zero.

Customer POV Behavior: Unlike naive applications that present a raw server error, the front-end handles this gracefully. It displays an alert indicating *"Item Not Deliverable to Your Location"* along with a *"Notify Me when Available at this Address"* button.

[PASTE SNAPSHOT 3: ITEM NOT DELIVERABLE CUSTOMER DRAWER]

HOW TO CAPTURE:
- In Customer View, click **"Buy Now"** on the OnePlus 11R card.
- Capture the modal drawer showing: *"Item Not Deliverable to Your Location: OnePlus 11R 5G is currently not deliverable to your selected address (VIT Vellore Campus) because stock is depleted at your nearest fulfillment dark store."*
- Paste the screenshot in this box.

Figure 4: Graceful Customer Stockout Experience ("Item Not Deliverable" & Waitlist Alert)

Step 4: Server Runtime Outage & Root Cause Stack Trace

While the customer is insulated with a friendly stockout notice, the underlying backend microservice encountered an unhandled exception. In the checkout calculation routine (`app/app.py`), the legacy inventory allocation formula divides total volume by remaining stock:

```
# app/app.py - Faulty Calculation Routine
discount_ratio = 100 / product["stock"]  # When stock == 0, throws
ZeroDivisionError
```

Because `product["stock"] == 0`, Python raises an unhandled `ZeroDivisionError: division by zero`, emitting a critical structured log message containing the full stack trace to standard output.

[PASTE SNAPSHOT 4: RUNTIME SERVER STACK TRACE]

HOW TO CAPTURE:
- Open your PowerShell or terminal window where `python app.py` is running.
- Locate the log lines showing:
`[CRITICAL] [SERVICE: watchdog-target-app] [INCIDENT_DETECTED] Type=ZeroDivisionError | Message=division by zero`
`Traceback (most recent call last): line 118, in checkout: _ = 100 / product["stock"]`
- Take a screenshot of this terminal output and paste it here.

Figure 5: CloudWatch Standard Output Stack Trace and Incident Telemetry Marker

Step 5: SRE Control Center & Cloud Telemetry Inspection

The Site Reliability Engineer switches to the **Developer & SRE Backstage View** on the application portal to analyze live telemetry across the AWS cloud infrastructure.

[PASTE SNAPSHOT 5: SRE CONTROL CENTER & ARCHITECTURE DASHBOARD]

HOW TO CAPTURE:
- At the top of `http://127.0.0.1:5000/`, click the button: **"2. Developer & SRE Backstage"**.
- Capture the screen showing the 3 metrics cards (Cloud Monitoring, Forensic S3 Vault, Agile Backlog Board) and the embedded System Architecture Diagram.
- Paste the screenshot in this box.

Figure 6: Developer & SRE Backstage Console Displaying Live Infrastructure Telemetry

Step 6: Amazon S3 Cloud Forensic Archival

Immediately upon log emission, Amazon CloudWatch activates the Log Subscription Filter. The filter invokes the **AWS Lambda Watchdog** (`watchdog-incident-creator`). The Watchdog parses the gzipped payload, extracts the error type, function name, and line number, and archives an immutable JSON crash dump to Amazon S3.

[PASTE SNAPSHOT 6: AMAZON S3 CRASH DUMP VAULT]

HOW TO CAPTURE:
- Open your AWS Console > Amazon S3.
- Navigate to bucket: `devops-incident-watchdog-logs-401537512022`.
- Open folder: `incidents/2026-10-14/` and capture the file listing showing `INC-1791945923-07CD12B5.json` (369 bytes).
- (Alternatively: Capture the terminal command `python scripts/check_aws.py` showing S3 bucket contents).
- Paste the screenshot in this box.

Figure 7: Tamper-Proof Incident Crash Dump Persisted in Private Amazon S3 Storage

Step 7: Automated Agile Bug User Story Creation on GitHub Issues

Simultaneously with S3 archival, the AWS Lambda Watchdog issues an authenticated HTTP POST request to the GitHub REST API (`/repos/hemalexe/devopss-incident-watchdog/issues`). It converts the technical crash telemetry into a formalized Agile Bug User Story with an actionable Definition of Done.

[PASTE SNAPSHOT 7: GITHUB ISSUE #1 WITH USER STORY]

HOW TO CAPTURE:
- Open your browser to: `https://github.com/hemalexe/devopss-incident-watchdog/issues/1`
- Capture the issue body showing:
- Title: *"Automated Incident Report: DevOps Incident Watchdog"*
- Agile User Story (*"As an Agile Team member, I need to..."*)
- Diagnostics Table with Incident ID and S3 URI.
- Acceptance Criteria Checklist.
- Paste the screenshot in this box.

Figure 8: Auto-Generated Agile Bug User Story on GitHub Issues Sprint Backlog

Step 8: Developer Code Patch & Smart Commit (`Fixes #1`)

The assigned developer inspects the Acceptance Criteria and implements defensive boundary checking in `app/app.py`:

```
# Corrected defensive implementation in app/app.py
if product["stock"] <= 0:
    logger.warning(f"Product '{product['name']}' out of stock.")
    return jsonify({
        "status": "not_deliverable",
        "message": f"Sorry, '{product['name']}' is out of stock at your dark
store.",
        "product": product["name"]
    }), 400
```

The developer commits the patch using Git Smart Commit syntax: `git commit -m "Fix zero-stock dark store allocation. Fixes #1"` and pushes to `origin/main`.

Step 9: GitHub Actions CI/CD Pipeline Execution & Automated Rollout

The push event automatically triggers the GitHub Actions CI/CD pipeline. The pipeline launches a clean virtual environment, installs project dependencies, executes 9 automated `pytest` unit tests, verifies regression safety, and deploys the updated code package to AWS Lambda.

[PASTE SNAPSHOT 9: GITHUB ACTIONS CI/CD GREEN RUN]

HOW TO CAPTURE:
- Open: `https://github.com/hemalexe/devopss-incident-watchdog/actions`
- Click on the latest workflow run: *"Agile DevOps CI/CD Pipeline"*.
- Capture the green checkmark showing all test jobs passed (9 passed in 0.53s).
- Paste the screenshot in this box.

Figure 9: Automated Regression Test Suite and Continuous Deployment Passing in GitHub Actions

Step 10: Zero-Touch Ticket Resolution & Lifecycle Completion

Because the developer referenced `Fixes #1` in the verified commit, GitHub automatically transitions Issue #1 from **Open** to **Closed (Done)** without manual intervention. Full traceability from initial production outage to sprint resolution is achieved.

[PASTE SNAPSHOT 10: GITHUB ISSUE CLOSED AS DONE]

HOW TO CAPTURE:
- Open: `https://github.com/hemalexe/devopss-incident-watchdog/issues/1`
- Scroll to the bottom of the issue thread.
- Capture the purple **"Closed"** badge and the commit reference: *"closed this in commit ... via Fixes #1"*.
- Paste the screenshot in this box.

Figure 10: Closed GitHub Issue Demonstrating Zero-Touch Agile Traceability

4. KEY PERFORMANCE INDICATORS & COMPARISON

Metric / Dimension	Traditional Manual Triage	DevOps Incident Watchdog
Mean-Time-To-Detect (MTTD)	15 – 45 Minutes (customer complaints)	< 5 Seconds (CloudWatch Log Filter)
Mean-Time-To-Acknowledge (MTTA)	2 – 8 Hours (manual ticket creation)	< 30 Seconds (Automated GitHub Issue)
Forensic Completeness	Truncated, copy-pasted terminal logs	100% full stack trace archived in Amazon S3
Agile Alignment	Manual backlog filing; often forgotten	Automated User Story with Definition of Done
Sprint Traceability	Disconnected tickets and code reviews	Zero-touch Smart Commits linking ticket to CI/CD

5. FACULTY VIVA VOCE EXAMINATION DEFENSE (Q&A)

Q1: Why use Amazon S3 when CloudWatch already stores logs?
Answer: CloudWatch log groups have configured retention limits and can be rotated, purged, or scale to zero. Amazon S3 provides an immutable, highly durable external audit vault essential for regulatory compliance, post-mortem Root Cause Analysis (RCA), and legal auditability without incurring heavy CloudWatch query fees.

Q2: How does GitHub receive the ticket without human intervention?
Answer: The AWS Lambda Watchdog executes upon CloudWatch log pattern matching, decodes the base64 event payload, extracts the stack trace, and issues an authenticated HTTP POST request to the GitHub REST API v3 endpoint (`/repos/.../issues`) containing the structured User Story and Acceptance Criteria.

Q3: How does this satisfy the "Agile" requirement of the course?
Answer: SRE monitoring usually stops at sending an email or PagerDuty buzz. This project bridges operations directly into Agile project management by transforming production defects into sprint backlog items structured as standard Agile User Stories with explicit Definition of Done checklists.

Q4: How does the ticket close automatically?
Answer: Through GitHub Smart Commit parsing: when the developer includes `Fixes #1` in the commit message, GitHub Actions executes regression tests. Upon passing, GitHub automatically links the commit SHA to Issue #1 and transitions the issue state to Done.
